

Python: warm-up exercises on iteration

This notebook was generated in **solutions** mode.

Exercise 1: Coin Flipping Simulation

Exercise Description

Write a piece of code that simulates coin tosses and keeps track of heads and tails.

Requirements:

- Use a variable `n` to specify the number of coin tosses
- Keep track of the number of heads and tails
- Use `random.randint(1, 100)` to simulate coin tosses (>50 = heads, ≤ 50 = tails)
- Store the final counts in variables `heads` and `tails`
- Return the variables `heads` and `tails`

Your solution

```
import random

def simulate_coin_flips(n):
    heads = 0
    tails = 0
    for _ in range(n):
        x = random.randint(1, 100)
        if x > 50:
            heads += 1
        else:
            tails += 1
    return heads, tails
```

Test your solution

```
# Test that heads + tails equals total number of tosses
n = 10
heads, tails = simulate_coin_flips(n)
total_tosses = heads + tails
assert total_tosses == n, f"Expected {n} total tosses, got {total_tosses}"
```

```
assert heads >= 0 and tails >= 0, 'Counts should be non-negative'
print('Test passed: coin flipping simulation works correctly!')
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 2: Count Odd and Even Numbers

Exercise Description

Write a piece of code that counts the number of odd and even numbers in a given range.

Requirements:

- Count numbers in the range from 1 to n (inclusive)
- Count how many numbers are odd and how many are even
- Store the counts in variables `odd_count` and `even_count`
- Return the variables `odd_count` and `even_count`

Your solution

```
def count_odd_even(n):  
    odd_count = 0  
    even_count = 0  
    for i in range(1, n + 1):  
        if i % 2 == 0:  
            even_count += 1  
        else:  
            odd_count += 1  
    return odd_count, even_count
```

Test your solution

```
n = 10  
odd_count, even_count = count_odd_even(n)  
assert odd_count == 5, f"Expected 5 odd numbers, got {odd_count}"  
assert even_count == 5, f"Expected 5 even numbers, got {even_count}"  
assert odd_count + even_count == n, f"Total count should equal {n}"  
print('Test passed: odd/even counting works correctly!')
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 3: Generate Fibonacci Sequence

Exercise Description

Write a piece of code that generates the first n numbers of the Fibonacci sequence.

Requirements:

- Generate and print the first 7 Fibonacci numbers
- The Fibonacci sequence starts with 0 and 1, then each subsequent number is the sum of the two preceding ones
- Print each number on a separate line as it's generated
- For n=7, the sequence should print: 0, 1, 1, 2, 3, 5, 8 (each on its own line)

Definition: Each number in the Fibonacci sequence is given by the sum of the two preceding ones. The Fibonacci sequence starts with two 'seed' numbers 0 and 1) and then the proper sequence starts: 0, 1, 1, 2, 3, 5, 8, 13, 21, ...

Your solution

```
def fibonacci(n):  
    if n >= 1:  
        print(0)  
    if n >= 2:
```

```

    print(1)

    # Use variables to track the last two numbers
    if n >= 3:
        prev_prev = 0 # F(0)
        prev = 1 # F(1)

        for i in range(2, n):
            current = prev_prev + prev
            print(current)
            # Update for next iteration
            prev_prev = prev
            prev = current

```

Test your solution

```

import io
import sys

# Capture the printed output
captured_output = io.StringIO()
sys.stdout = captured_output

n = 7
fibonacci(n)

# Restore normal output
sys.stdout = sys.__stdout__

# Get the output and convert to list of numbers
output_lines = captured_output.getvalue().strip().split('\n')

```

```
fibonacci_numbers = [int(line) for line in output_lines if line.strip()]

expected_sequence = [0, 1, 1, 2, 3, 5, 8]
assert fibonacci_numbers == expected_sequence, f"Expected {expected_sequence}, got {fibonacci_numbers}"
assert len(fibonacci_numbers) == n, f"Expected {n} numbers, got {len(fibonacci_numbers)}"
print('Test passed: Fibonacci sequence generation works correctly!')
print(f"Generated sequence: {fibonacci_numbers}")
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 4: Multiples of 3 and 5

Exercise Description

Write a piece of code that processes numbers from 1 to n and prints classification based on multiples.

Requirements:

- Process integers from 1 to n
- For multiples of both 3 and 5, print 'Mult35'
- For multiples of 3 only, print 'Mult3'
- For multiples of 5 only, print 'Mult5'
- For other numbers, print the number itself
- Print each result on a separate line

Your solution

```
def classify_multiples(n):  
    for num in range(1, n + 1):  
        if num % 3 == 0 and num % 5 == 0:  
            print('Mult35')  
        elif num % 3 == 0:  
            print('Mult3')  
        elif num % 5 == 0:  
            print('Mult5')  
        else:  
            print(num)
```

Test your solution

```
import io  
import sys  
  
# Capture the printed output  
captured_output = io.StringIO()  
sys.stdout = captured_output
```



```

n = 15
classify_multiples(n)

# Restore normal output
sys.stdout = sys.__stdout__

# Get the output and convert to list for testing
output_lines = captured_output.getvalue().strip().split('\n')
result = []
for line in output_lines:
    if line.strip().isdigit():
        result.append(int(line.strip()))
    else:
        result.append(line.strip())

# Test specific positions (convert to 0-based indexing)
assert result[14] == 'Mult35', f"Expected 'Mult35' at position 14, got {result[14]}"
assert result[2] == 'Mult3', f"Expected 'Mult3' at position 2, got {result[2]}"
assert result[4] == 'Mult5', f"Expected 'Mult5' at position 4, got {result[4]}"
assert len(result) == n, f"Expected {n} elements, got {len(result)}"
print('Test passed: multiples classification works correctly!')
print(f"Generated output: {result[:10]}...") # Show first 10 elements

```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 5: Print Number Subset

Exercise Description

Write a piece of code that scans through a range of numbers but only prints a subset.

Requirements:

- Scan all integers from 1 to n
- Only print numbers from 1 to m (where $m \leq n$)
- Print each number on a separate line

Your solution

```
def collect_subset(n, m):  
    for num in range(1, n + 1):  
        if num <= m:  
            print(num)
```

Test your solution

```
import io  
import sys
```

```

# Capture the printed output
captured_output = io.StringIO()
sys.stdout = captured_output

n = 10
m = 6
collect_subset(n, m)

# Restore normal output
sys.stdout = sys.__stdout__

# Get the output and convert to list of integers for testing
output_lines = captured_output.getvalue().strip().split('\n')
subset_numbers = [int(line.strip()) for line in output_lines if line.strip()]

expected_subset = [1, 2, 3, 4, 5, 6]
assert subset_numbers == expected_subset, f"Expected {expected_subset}, got {subset_r
assert len(subset_numbers) == m, f"Expected {m} numbers, got {len(subset_numbers)}"
assert max(subset_numbers) == m, f"Expected max value {m}, got {max(subset_numbers)}"
print('Test passed: subset collection works correctly!')
print(f"Generated subset: {subset_numbers}")

```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 6: Find Prime Numbers

Exercise Description

Write a piece of code that finds all prime numbers smaller than a given number n .

Requirements:

- Find all prime numbers smaller than n
- A prime number is greater than 1 and has no divisors other than 1 and itself
- Print each prime number on a separate line as it's found
- Use a simple primality test: check if a number is divisible by any number from 2 to $n/2$

Your solution

```
def primes_less_than(n):  
    for num in range(2, n):  
        is_prime = True  
        for d in range(2, num // 2 + 1):  
            if num % d == 0:  
                is_prime = False  
                break  
        if is_prime:  
            print(num)
```

```
if is_prime:  
    print(num)
```

Test your solution

```
import io  
import sys  
  
# Capture the printed output  
captured_output = io.StringIO()  
sys.stdout = captured_output  
  
n = 15  
primes_less_than(n)  
  
# Restore normal output  
sys.stdout = sys.__stdout__  
  
# Get the output and convert to list of integers for testing  
output_lines = captured_output.getvalue().strip().split('\n')  
prime_numbers = [int(line.strip()) for line in output_lines if line.strip()]  
  
expected_primes = [2, 3, 5, 7, 11, 13]  
assert prime_numbers == expected_primes, f"Expected {expected_primes}, got {prime_numbers}"  
assert all(p < n for p in prime_numbers), 'All primes should be less than n'  
print('Test passed: prime number finding works correctly!')  
print(f"Found primes: {prime_numbers}")
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 7: Find Pythagorean Triples

Exercise Description

Write a piece of code that finds all Pythagorean triples for a given number n .

Requirements:

- Find all pairs (x, y) such that $x^2 + y^2 = n^2$
- Both x and y should be positive integers less than n
- Print each valid pair in the format " x y " (separated by a space)
- A Pythagorean triple consists of three positive integers x , y , and n such that $x^2 + y^2 = n^2$

Your solution

```
def pythagorean_pairs(n):  
    for x in range(1, n):  
        for y in range(1, n):
```

```
if x**2 + y**2 == n**2:
    print(x, y)
```

Test your solution

```
import io
import sys

# Capture the printed output
captured_output = io.StringIO()
sys.stdout = captured_output

n = 5
pythagorean_pairs(n)

# Restore normal output
sys.stdout = sys.__stdout__

# Get the output and convert to list of tuples for testing
output_lines = captured_output.getvalue().strip().split('\n')
pythagorean_pairs_result = []
for line in output_lines:
    if line.strip():
        x, y = map(int, line.strip().split())
        pythagorean_pairs_result.append((x, y))

expected_pairs = [(3, 4), (4, 3)]
assert pythagorean_pairs_result == expected_pairs, f"Expected {expected_pairs}, got {pythagorean_pairs_result}"
for x, y in pythagorean_pairs_result:
    assert x**2 + y**2 == n**2
```

